

Workflow Validation and Interface Checks — BDIP Contribute

Objective

This document describes all validations, automatic checks, navigation restrictions, workflow states, and visual feedback mechanisms implemented in the BDIP Contribute interface workflow.

| Note: this document does not include low-level internal code validations or implementation-specific technical details.

1. Global Workflow Validations

1.1. Unsaved Changes Protection (Dirty State)

The system tracks unsaved changes in the following sections:

- Dockerfile
- README
- Metadata
- Ontology
- Test Instructions

Implemented Checks

Current State vs Saved State Comparison

Whenever the current content differs from the latest saved version, the section is marked as `"dirty"`.

Navigation Blocking

When the user attempts to change pages:

- the system checks whether unsaved changes exist
- if unsaved changes are detected, a confirmation dialog is displayed
- navigation is blocked until the user decides

Workflow:

- `"Yes, leave without saving"` → discards changes
- `"Cancel"` → keeps the user on the current page

1.2. Autosave During Sidebar Navigation

When navigating through the sidebar, the system automatically attempts to save changes in each section.

Special JSON Metadata Validation

If the JSON format is invalid:

- autosave is blocked
- an error is displayed to the user
- navigation is interrupted

```
[ERROR]: Cannot autosave Metadata because the JSON is invalid.
```

1.3. Global Data Reset

A strong protection mechanism exists before deleting all projects.

Required Confirmation

The user must type exactly:

```
DELETE
```

Behavior

- While the input does not exactly match `DELETE`, the `"Delete All"` button remains disabled
- Only after confirmation does the system:
 - remove all projects
 - delete the local database
 - reset the workflow

1.4. Protected Navigation

In almost all pages:

Mandatory Validation

The system verifies whether an active project is selected.

If no project is selected, the workflow is interrupted:

```
[WARNING]: Select a project first!
```

2. Home Page

2.1. Workflow Selection

The user can:

- Create a new project
- Test an existing Docker image
- Download available input test data

3. Create Project

3.1. Project Type Selection

Available project types:

- Regular
- From Image (Dockerfile)
- From Image (No Dockerfile)
- Update

Conditional Behavior

The selected project type dynamically changes:

- available workflow

- Dockerfile rules
 - future validations
 - mandatory sections
-

3.2. New Project Creation

Implemented Validations

1. Empty Project Name

If the project name is empty, project creation is blocked:

```
[ERROR]: Please enter a name.
```

2. Existing Project Validation

The system verifies whether the project already exists:

- locally
- on GitHub

If the project already exists:

```
[WARNING]: This project already exists!
```

3.3. "Update Project" Workflow

Image Search

- Only allows creating new versions from existing images already available in the repository

New Version Creation

1. Empty Version

If the version field is empty, the workflow is interrupted:

```
[ERROR]: Enter a version name.
```

2. Existing Version

The system validates whether the version already exists in:

- latest
- recommended
- no_longer_tested

If the version already exists:

```
[ERROR]: Version [X.X.X] already exists for [selected_project]
```

4. Current Project

4.1. Automatic Project Status

The system automatically calculates the status of each section.

Manual editing is also allowed:

- DONE
- NOT DONE
- IN PROGRESS
- REVIEW

Note: the workflow dynamically changes according to the project type.

4.2. Submission Package Generation

The submission button is only enabled when:

All steps == DONE

5. Dockerfile Page

In this section, the user creates the Dockerfile and may also attempt to build the Docker image.

5.1. General Checks

License Warning

The system displays a mandatory warning regarding:

- redistribution
- licensing
- containerization permissions

5.2. "FROM IMAGE WITHOUT DOCKERFILE" Workflow

This project type:

- does not require a Dockerfile
- completely blocks this step

The user only receives contextual information:

This project does not require a Dockerfile.

5.3. "FROM IMAGE WITH DOCKERFILE" Workflow

The system only allows:

FROM image

Implemented Validations

Only FROM Instruction Allowed

[ERROR]: Only FROM instruction is allowed.

5.4. "REGULAR / UPDATE" Workflow

Save Progress

The `SAVE PROGRESS` button is only enabled when unsaved changes exist.

5.5. "UPDATE" Workflow

GitHub Repository Validation

The system validates whether a Dockerfile exists in the repository:

```
[WARNING]: No Dockerfile found in repository.
```

Repository Structure Validation

The repository must follow the expected structure:

```
/<project>/<version>/Dockerfile
```

Possible warnings:

```
[WARNING]: This project does not follow the expected version structure (missing version folder).
```

or

```
[WARNING]: The Dockerfile is named 'dockerfile' (lowercase). It should be 'Dockerfile'.
```

5.6. License Agreement

Before saving or validating:

- the user must confirm the license agreement checkbox

If not confirmed:

```
[ERROR]: Please confirm the license agreement before saving.
```

or

```
[ERROR]: Please confirm the license agreement before finishing.
```

5.7. Dockerfile Validations

Empty Dockerfile

```
[ERROR]: Dockerfile is empty.
```

Mandatory FROM Instruction

```
[ERROR]: Missing FROM instruction.
```

Hadolint Integration

The system integrates Hadolint and dynamically ignores rules configured in the GitHub configuration file.

Validation Result

Validation Failure

If errors exist:

- validation fails
- the Dockerfile is not saved locally

```
[ERROR]: Validation failed!
```

Validation Success

If only warnings exist or no issues are found:

- validation succeeds
- Dockerfile is saved locally
- status is updated to DONE

```
[WARNING]: Validation passed with warnings!
```

or

```
[SUCCESS]: No issues found!
```

5.8. Build Docker Image

Before building the image, the system verifies whether the Dockerfile exists locally.

If not found:

```
[ERROR]: Dockerfile not found.
```

Results

SUCCESS

- `build_success == True`
- status updated to DONE

```
[SUCCESS]: Docker image built successfully!
```

FAILURE

A detailed execution log is displayed to the user:

```
[ERROR]: Build failed.
```

6. README Page

In this section, the user creates the README using a structured form.

6.1. General README Validations

The system performs validation based on:

- required fields

- optional fields

Conditional Fields

Some fields contain conditional logic.

If a dependent field is filled, another field automatically becomes mandatory.

```
If X exists -> Y becomes mandatory
```

Note: some fields are automatically populated from the metadata (if empty), but users may still modify them if needed (e.g. `tool_name`, `tool_url`, `tool_url_help`).

6.2. "FROM IMAGE WITHOUT DOCKERFILE" Workflow

The system attempts to fetch and download the README from DockerHub.

If the request fails:

```
[ERROR]: Failed to fetch README.
```

6.3. "UPDATE" Workflow

README Repository Validation

The system checks whether a README exists in the GitHub repository:

```
[WARNING]: No README found in repository.
```

Repository Structure Validation

The repository must follow:

```
/<project>/<version>/README.md
```

If invalid:

```
[WARNING]: This project does not follow the expected version structure (missing version folder).
```

6.4. README Content Validation

Empty README Validation

```
[ERROR]: README is empty!
```

Required Fields Validation

The system verifies whether all mandatory () fields are completed.

Validation Result

Failure

If errors exist:

```
[ERROR]: README validation failed!
```

Success

If warnings exist or no issues are found:

```
[WARNING]: README valid with warnings!
```

or

```
[SUCCESS]: README valid!
```

7. Metadata Page

In this section, the user creates the project-specific `metadata.json` file using a structured form.

7.1. "UPDATE" Workflow

For update projects:

- metadata is automatically loaded from GitHub
- users may only visualize the existing metadata

If no metadata exists:

```
[WARNING]: No metadata found.
```

7.2. General Metadata Validations

The validation workflow includes:

- required fields
- optional fields

Conditional Fields

Some fields use conditional logic:

```
If X exists -> Y becomes mandatory
```

- Note 1: some fields are automatically populated from the README (if empty), but users may still edit them if needed (e.g. `tool_name`, `manual_url`, `source_url`).
- Note 2: some fields are automatically generated (e.g. `status`, `test_data_url`, `test_results_url`).

7.3. JSON Format Validation

The system validates whether the JSON structure is valid:

```
[ERROR]: Cannot save – JSON is invalid. Please fix syntax errors before saving.
```

8. Ontology Page

In this section, the user selects ontology terms associated with the project.

8.1. Important Considerations

- The ontology is loaded directly from GitHub
- Users may filter ontology terms using partial search
- Only terminal ontology terms can be selected
- At least one ontology term must be selected
- All selected ontology IDs must exist in the `dio.obo` ontology file available on GitHub

8.2. "UPDATE" Workflow

- Users may only visualize ontology terms
- Only ontology terms associated with the selected project are displayed

8.3. New Ontology Terms

Users may suggest new ontology terms.

These suggestions are later reviewed by project managers.

8.4. AI Ontology Assistant

The system automatically generates a prompt containing:

- the project description (based on the README)
- the list of available ontology terms

Objective

Assist users in selecting the most appropriate ontology terms.

9. Build and Test

In this section, users can build and test Docker images to ensure correct execution.

9.1. "FROM IMAGE WITHOUT DOCKERFILE" Workflow

This project type does not allow building or testing images.

```
[WARNING]: No Build/Test Available for this Project Type.  
This project uses an existing Docker image and does not include a Dockerfile.
```

The system only provides a text area for execution instructions.

Validation

Instructions cannot be empty:

```
[ERROR]: Instructions cannot be empty.
```

9.2. "UPDATE" Workflow

- Verifies whether additional repository files required for building exist
- Tests the image using test data URLs defined in the metadata

9.3. General Validations

Build Image Phase

- verifies whether a Dockerfile exists

Test Image Phase

- verifies whether input files exist
- verifies whether a test command (`docker run ...`) exists

| Note: the system generates a complete execution log to assist the user.

10. Submission Packaging

This is the final workflow step.

All previous required steps must be completed before generating the `/for_submission` folder.

The folder is only generated when all required steps are marked as:

DONE

Only projects containing a valid `/for_submission` folder can be validated by project managers through:

Home > Test Docker Image